

File Handling and Stream I/O in C

[DEF] Definition to Remember

File Input/Output in C provides persistent data storage by transferring data between primary memory and secondary storage devices using FILE pointers and buffer streams.

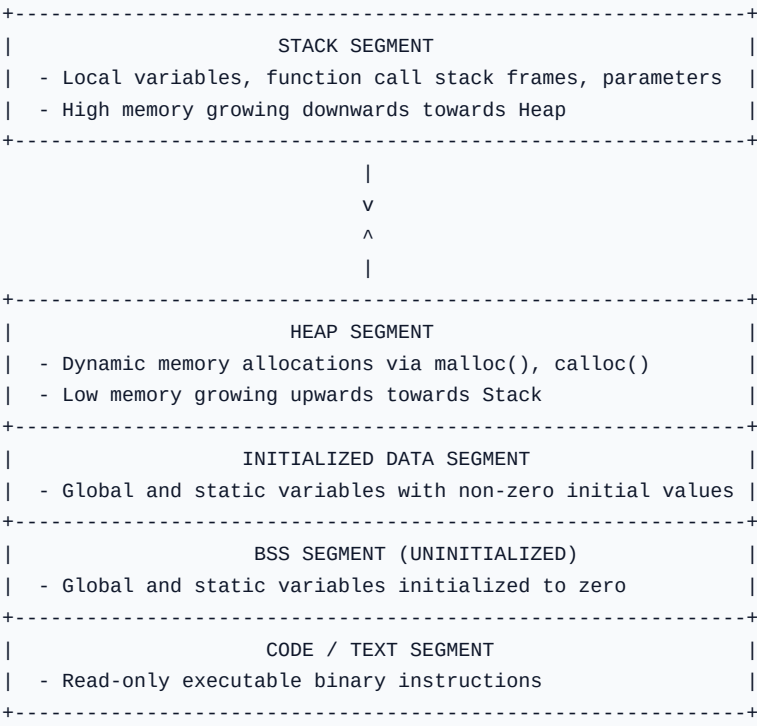
1. Concept Overview & Fundamentals

File Handling and Stream I/O in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization



Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers	Stack: O(1) push/pop; Heap: O(1)
Type Checking	Static Typing	Compiler verifies types during compila	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memor	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops,	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: File InputOutput\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- * Operating System Kernels: Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

* Em

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: append log

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu

int ma
/*
*
*
*
*
*
*
*
FI
ch
if
}
pr
ge
fg
```

```
fp
fo
re
}
```

Problem 2: check file open

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu

int ma
/*
*
*
*
*
*
*
*

ch

pr
sc

FI

if
}

}

re
}
```

Problem 3: copy file

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu

int ma
/*
*
*
*
*
*
*
*

FI
FI
```

```
ch
if
}
wh
}
pr
fo
fo
re
}
```

Problem 4: read write file

Problem Description & Objective:

C Code Implementation:

Implem

```
#inclu
int ma
/*
*
*
*
*
*
*
FI
ch
if
}
//
pr
fg
fp
//
re
//
pr
wh
}
fo
re
}
```

Problem 5: sum from file

Problem Description & Objective:

C Code Implementation:

Implem

```
#include <stdio.h>

int main()
/*
 *
 *
 *
 *
 *
 *
 */
{
    FILE *f;
    int sum = 0;

    if (f = fopen("sum.txt", "r"))
    {
        while (fscanf(f, "%d", &sum) != EOF)
        {
            sum += sum;
        }
        printf("Sum: %d\n", sum);
        fclose(f);
    }
}
```

Problem 6: write lines

Problem Description & Objective:

C Code Implementation:

Implem

```
#include <stdio.h>

int main()
/*
 *
 *
 *
 *
 *
 *
 */
{
    FILE *f;
    char line[100];

    if (f = fopen("write.txt", "w"))
    {
        printf("Enter lines to write: ");
        while (fgets(line, 100, stdin) != NULL)
        {
            fputs(line, f);
        }
        fclose(f);
    }
}
```

sc
ge

fo

}
fo
re
}

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. FILE pointer: FILE* fp tracks the stream state, buffer, and current read/write offset.
2. File modes: 'r' (read), 'w' (overwrite/create), 'a' (append), 'r+' (read/write), 'b' (binary mode).
3. Always verify fp != NULL after fopen(); missing checks cause crash on non-existent files.
4. Standard file I/O: fgetc/fputc, fgets/fputs, fprintf/fscanf, fread/fwrite.
5. Always close files with fclose(fp) to flush output buffers and release OS file descriptors.

[RECALL] Quick Recall

fopen(mode) -> Check NULL -> fprintf / fscanf / fgets -> fflush -> fclose(fp)